In 1993, Udi Manber and Gene Myers introduced **suffix arrays** as a memory-efficient alternative to suffix trees. To construct *SuffixArray*(*Text*), we first sort all suffixes of *Text* lexicographically, assuming that `"$"` comes first in the alphabet. For example, below is the sorted list of suffixes of *Text* = `"panamabananas$"`, along with their starting positions in *Text*.

Starting Positions	Sorted Suffixes
13	$
5	abananas$
3	amabananas$
1	anamabananas$
7	ananas$
9	anas$
11	as$
6	bananas$
4	mabananas$
2	namabananas$
8	nanas$
10	nas$
0	panamabananas$
12	s$

The suffix array is the list of starting positions of these sorted suffixes:

SuffixArray(`"panamabananas$"`) = (13, 5, 3, 1, 7, 9, 11, 6, 4, 2, 8, 10, 0, 12).

Suffix Array Construction Problem: *Construct the suffix array of a string.*

Input: A string *Text*.

Output: *SuffixArray*(*Text*).

CODE CHALLENGE: Solve the Suffix Array Construction Problem.

Sample Input:

```
AACGATAGCGGTAGA$
```

Sample Output:

```
15, 14, 0, 1, 12, 6, 4, 2, 8, 13, 3, 7, 9, 10, 11, 5
```

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/SuffixArray.txt)

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-310/step/2

The Suffix Array Construction Problem can easily be solved after sorting all suffixes of *Text*, but since even the fastest algorithms for sorting an array of n elements require $O(n \cdot \log n)$ comparisons, sorting all suffixes takes $O(|Text| \cdot \log (|Text|))$ comparisons. However, there exists a faster algorithm that constructs suffix arrays in linear time and requires only about a fifth as much memory as suffix trees, which knocks the 60 GB memory requirement for the human genome down to 12 GB.

STOP and Think: Given a suffix tree, can you quickly transform it into a suffix array? Given a suffix array, can you quickly transform it into a suffix tree?

As the preceding question suggests, suffix arrays and suffix trees are practically equivalent: every algorithm using suffix arrays can be translated into an algorithm using suffix trees, and vice-versa (see DETOUR: Suffix Arrays and Suffix Trees (/lesson/lesson-308/)).

Genome compression

Suffix arrays have greatly reduced the memory required for efficient text searches, and until the start of this century, they represented the state of the art in pattern matching. Can we be so ambitious as to look for a data structure that would encode Text using memory approximately equal to the length of Text while still enabling fast pattern matching?

To approach this question, we will digress to consider the superficially unrelated topic of **text compression**. In one simple compression technique called **run-length encoding**, we replace a **run** of *k* consecutive occurrences of symbol *s* with only two symbols: *k*, followed by *s*. For example, run-length encoding would compress `TTTTTGGGAAAACCCCCCA` into `5T3G4A6C1A`.

Run-length encoding works well for strings having lots of long runs, but genomes do not have many runs. What they do have is repeats, as we learned when assembling genomes. It would therefore be nice if we could first manipulate the genome to convert repeats into runs and then apply run-length encoding to the resulting string.

A naive way of creating runs in a string is to reorder the string's symbols lexicographically. For example, `TACGTAACGATACGAT` would become `AAAAACCCGGGTTTT`, which we could then compress into `5A3C3G4T`. This method would represent a 3 GB human genome file using just four numbers.

STOP and Think: What is wrong with applying this compression method to genomes?

Ordering a string's symbols lexicographically is not suitable for compression because many different strings will get compressed into the *same* string. For example, `GCATCATGCAT` and `ACTGACTACTG` — as well as any string with the same nucleotide counts) — get reordered into `AAACCCGGTTT`. As a result, we cannot **decompress** the compressed string, i.e., invert the compression operation to produce the original string.

Constructing the Burrows-Wheeler transform

Let's consider a different method of converting the repeats of a string into runs that was proposed by Michael Burrows and David Wheeler in 1994. First, form all possible **cyclic rotations** of *Text*; a cyclic rotation is defined by chopping off a suffix from the end of *Text* and appending this suffix to the beginning of *Text*. Next — similarly to suffix arrays — order all the cyclic rotations of *Text* lexicographically to form a |*Text*| × |*Text*| matrix of symbols that we call the **Burrows-Wheeler matrix** and denote by *M*(*Text*). The figure below illustrates the construction of *M*(*Text*).

Cyclic Rotations	*M*("panamabananas$")
panamabananas$	$ p a n a m a b a n a n a s
$panamabananas	a b a n a n a s $ p a n a m
s$panamabanana	a m a b a n a n a s $ p a n
as$panamabanan	a n a m a b a n a n a s $ p
nas$panamabana	a n a n a s $ p a n a m a b
anas$panamaban	a n a s $ p a n a m a b a n
nanas$panamaba	a s $ p a n a m a b a n a n
ananas$panamab	b a n a n a s $ p a n a m a
bananas$panama	m a b a n a n a s $ p a n a
abananas$panam	n a m a b a n a n a s $ p a
mabananas$pana	n a n a s $ p a n a m a b a
amabananas$pan	n a s $ p a n a m a b a n a
namabananas$pa	p a n a m a b a n a n a s $
anamabananas$p	s $ p a n a m a b a n a n a

Figure: All cyclic rotations of "panamabananas$" (left) and the Burrows-Wheeler matrix *M*("panamabananas$") of all lexicographically ordered cyclic rotations (right).

Notice that the first column of *M*(*Text*) contains the symbols of *Text* ordered lexico- graphically, which is just the naive compression of Text that we already described. In turn, the second column of *M*(*Text*) contains the second symbols of all cyclic rotations of *Text*, and so it too represents a (different) rearrangement of symbols from *Text*. The same reasoning applies to show that any column of *M*(*Text*) is some rearrangement of the symbols of *Text*. We are interested in the last column of *M*(*Text*), called the **Burrows-Wheeler transform** of *Text*, or *BWT*(*Text*), which is shown in red in the figure below.

Cyclic Rotations	*M*("panamabananas$")
panamabananas$	$ p a n a m a b a n a n a **s**
$panamabananas	a b a n a n a s $ p a n a **m**
s$panamabanana	a m a b a n a n a s $ p a **n**
as$panamabanan	a n a m a b a n a n a s $ **p**
nas$panamabana	a n a n a s $ p a n a m a **b**
anas$panamaban	a n a s $ p a n a m a b a **n**
nanas$panamaba	a s $ p a n a m a b a n a **n**
ananas$panamab	b a n a n a s $ p a n a m **a**
bananas$panama	m a b a n a n a s $ p a n **a**
abananas$panam	n a m a b a n a n a s $ p **a**
mabananas$pana	n a n a s $ p a n a m a b **a**
amabananas$pan	n a s $ p a n a m a b a n **a**
namabananas$pa	p a n a m a b a n a n a s **$**
anamabananas$p	s $ p a n a m a b a n a n **a**

Figure: *BWT*("panamabananas$") is given by the last column of *M*("panamabananas$"): "smnpbnnaaaaa$a".

STOP and Think: We have seen that the first column of *M*(*Text*) cannot be uniquely decompressed to yield *Text*. Do you think that some other column of *M*(*Text*) can be inverted to yield *Text*?

Burrows-Wheeler Transform Construction Problem: *Construct the Burrows-Wheeler transform of a string.*

Input: A string *Text*.

Output: *BWT*(*Text*).

CODE CHALLENGE: Solve the Burrows-Wheeler Transform Construction Problem.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/BWT.txt)

Note: Although it is possible to construct the Burrows-Wheeler transform in O(|*Text*|) time and space, we do not expect you to implement such a fast algorithm. In other words, it is perfectly fine to produce BWT(*Text*) by first producing the complete Burrows-Wheeler matrix M(*Text*).

Sample Input:

```
GCGTGCCTGGTCA$
```

Sample Output:

```
ACTGGCT$TGCGGC
```

Time Limit: 5 mins

From repeats to runs

If we re-examine the Burrows-Wheeler transform of "panamabananas$, we immediately notice that it has created the run "aaaaa" in *BWT*("panamabananas$") = "smnpbnn**aaaaa**$a".

Cyclic Rotations	*M*("panamabananas$")
panamabananas$	$ p a n a m a b a n a n a **s**
$panamabananas	a b a n a n a s $ p a n a **m**
s$panamabanana	a m a b a n a n a s $ p a **n**
as$panamabanan	a n a m a b a n a n a s $ **p**
nas$panamabana	a n a n a s $ p a n a m a **b**
anas$panamaban	a n a s $ p a n a m a b a **n**
nanas$panamaba	a s $ p a n a m a b a n a **n**
ananas$panamab	b a n a n a s $ p a n a m **a**
bananas$panama	m a b a n a n a s $ p a n **a**
abananas$panam	n a m a b a n a n a s $ p **a**
mabananas$pana	n a n a s $ p a n a m a b **a**
amabananas$pan	n a s $ p a n a m a b a n **a**
namabananas$pa	p a n a m a b a n a n a s **$**
anamabananas$p	s $ p a n a m a b a n a n **a**

STOP and Think: Why do you think that the Burrows-Wheeler Transform produced this run?

Imagine that we take the Burrows-Wheeler transform of Watson and Crick's 1953 paper on the double helix structure of DNA. The word "and" is repeated often in English, which means that when we form all possible cyclic rotations of the Watson & Crick paper, we will witness a large number of rotations beginning with `"and..."` In turn, we will observe many rotations that begin with `"nd..."` and end with `"...a"`. When all the cyclic rotations of *Text* are sorted lexicographically to form *M*(*Text*), all rows that begin with `"nd..."` and end with `"...a"` will tend to clump together. As illustrated in the figure below, this clumping produces runs of `"a"` in the final column of *M*(*Text*), which we know is *BWT*(*Text*).

```
nd Corey (1).  They kindly made their manuscript availa ..... a
nd criticism, especially on interatomic distances.  We  ..... a
nd cytosine.  The sequence of bases on a single chain d ..... a
nd experimentally (3,4) that the ratio of the amounts o ..... u
nd for this reason we shall not comment on it.  We wish ..... a
nd guanine (purine) with cytosine (pyrimidine).  In oth ..... a
nd ideas of Dr.  M. H. F. Wilkins, Dr.  R. E. Franklin  ..... a
nd its water content is rather high.  At lower water co ..... a
nd pyrimidine bases.  The planes of the bases are perpe ..... a
nd stereochemical arguments.  It has not escaped our no ..... a
nd that only specific pairs of bases can bond together  ..... u
nd the atoms near it is close to Furberg's 'standard co ..... a
nd the bases on the inside, linked together by hydrogen ..... a
nd the bases on the outside.  In our opinion, this stru ..... a
nd the other a pyrimidine for bonding to occur.  The hy ..... a
nd the phosphates on the outside.  The configuration of ..... a
nd the ration of guanine to cytosine, are always very c ..... a
nd the same axis (see diagram).  We have made the usual ..... u
nd their co-workers at King's College, London.  One of  ..... a
```

Figure: A few consecutive rows selected from *M*(*Text*), where *Text* is Watson and Crick's 1953 paper on the double helix. Rows beginning with `"nd..."` often end with `"...a"` because of the common occurrence of the word `"and"` in English, which causes runs of `"a"` in *BWT*(*Text*).

The substring "ana" in "panamabananas$" plays the role of "and" in Watson and Crick's paper and explains three of the five occurrences of "a" in the repeat "aaaaa" in *BWT*("panamabananas$") = "smnpbnn**aaaaa**$a". When the Burrows-Wheeler transform is applied to a genome, it converts the genome's many repeats to runs. As we already suggested, after applying the Burrows-Wheeler transform, we can apply an additional compression method such as run-length encoding in order to further reduce the memory.

EXERCISE BREAK: There is only one run of length at least 10 in the *E. coli* genome. How many runs of length at least 10 do you find after applying the Burrows-Wheeler transform to the *E. coli* genome?

Download Genome (4.6 MB) (/media/attachments/lessons/4/E-coli.txt)

To solve this problem please visit stepic.org/lesson/-297/step/7

Before we get ahead of ourselves, remember that compressing a genome does not count for much if we cannot decompress it. In particular, if there exist a pair of genomes that the Burrows-Wheeler transform compresses into the same string, then we will not be able to decompress this string. But it turns out that the Burrows-Wheeler transform is reversible!

STOP and Think: Can you find the (unique) string whose Burrows-Wheeler transform is `"enwvpeoseu$llt"`? It could be `"newtloveslupe$"`, `"elevenplustwo$"`, `"unwellpesovet$"`, or something else entirely.

Consider the toy example of *BWT*(*Text*) = `"ard$rcaaaabb"`. First, recall that the first column of *M*(*Text*) is just the lexicographic rearrangement of symbols in *BWT*(*Text*), i.e., `"$aaaaabbcdrr"`. For convenience, we will henceforth use the shorthand *FirstColumn* and *LastColumn* (i.e., *BWT*(*Text*)) when referring to the first and last columns of *M*(*Text*), respectively.

We also know that the first row of *M*(*Text*) is the cyclic rotation of *Text* beginning with `"$"`, which occurs at the end of *Text*. Thus, if we determine the first row of *M*(*Text*), then we can move the `"$"` to the end of this row and reproduce *Tex*t. But how do we determine the remaining symbols in this first row, if all we know is *FirstColumn* and *LastColumn*?

```
$ ? ? ? ? ? ? ? ? ? a
a ? ? ? ? ? ? ? ? ? r
a ? ? ? ? ? ? ? ? ? d
a ? ? ? ? ? ? ? ? ? $
a ? ? ? ? ? ? ? ? ? r
a ? ? ? ? ? ? ? ? ? c
b ? ? ? ? ? ? ? ? ? a
b ? ? ? ? ? ? ? ? ? a
c ? ? ? ? ? ? ? ? ? a
d ? ? ? ? ? ? ? ? ? a
r ? ? ? ? ? ? ? ? ? b
r ? ? ? ? ? ? ? ? ? b
```

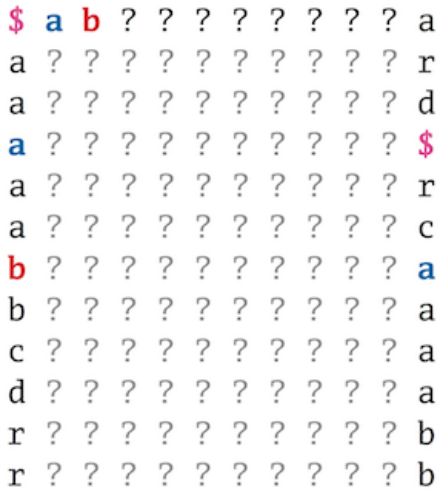
STOP and Think: Using the first and last columns of the Burrows-Wheeler matrix shown above, can you find the first symbol of *Text*?

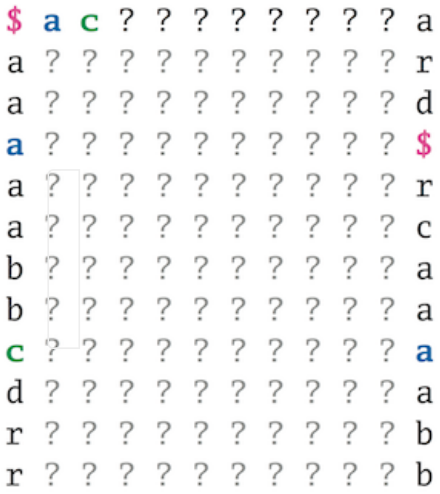
Note that the first symbol in *Text* must follow `"$"` in *any* cyclic rotation of *Text*. Because `"$"` occurs as the fourth symbol of *LastColumn* = `"ard$rcaaaabb"`, we know that if we walk one symbol to the right from the end of the fourth row of *M*(*Text*), then we will "wrap around" and arrive at the fourth symbol of *FirstColumn*, which is `"a"` in `"$aaaaabbcdrr"`. Therefore, this `"a"` belongs in the first position of *Text*:

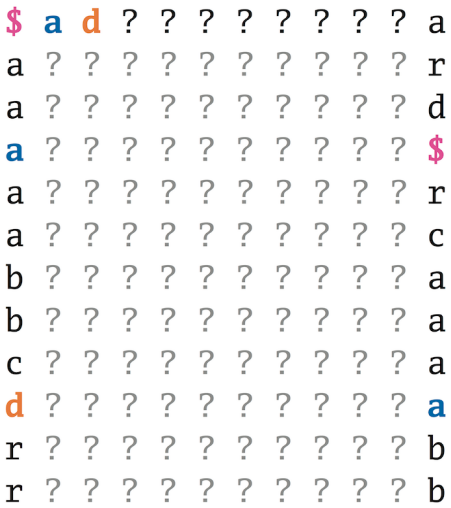
```
$ a ? ? ? ? ? ? ? ? a
a ? ? ? ? ? ? ? ? ? r
a ? ? ? ? ? ? ? ? ? d
a ? ? ? ? ? ? ? ? ? $
a ? ? ? ? ? ? ? ? ? r
a ? ? ? ? ? ? ? ? ? c
b ? ? ? ? ? ? ? ? ? a
b ? ? ? ? ? ? ? ? ? a
c ? ? ? ? ? ? ? ? ? a
d ? ? ? ? ? ? ? ? ? a
r ? ? ? ? ? ? ? ? ? b
r ? ? ? ? ? ? ? ? ? b
```

STOP and Think: Which symbol is hiding in the second position of *Text*?

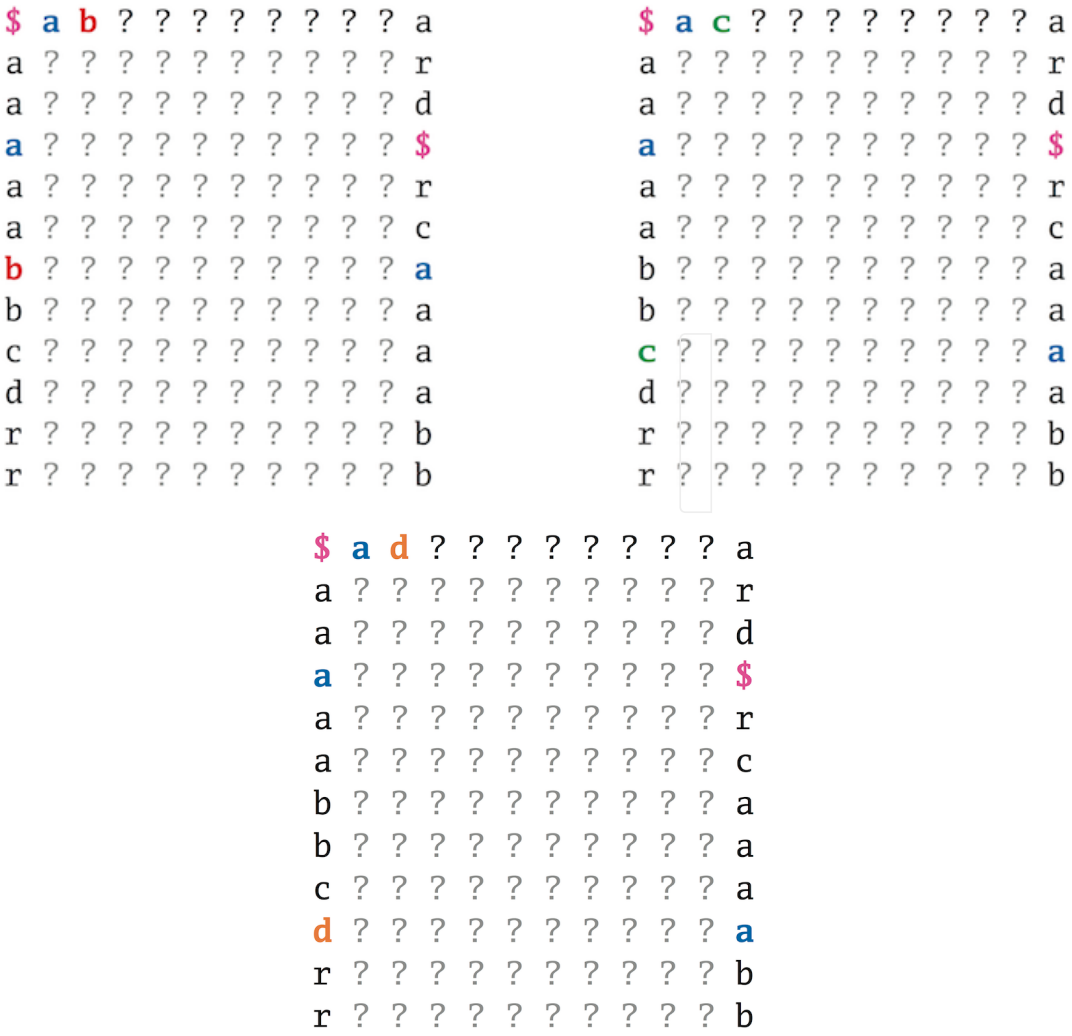
Following the same logic of “wrapping around”, the next symbol of Text should be the first symbol in a row of *M*(*Text*) that ends in `"a"`. The only trouble is that five rows end in `"a"`, and we don’t know which of them is the correct one! If we guess that this `"a"` is the seventh symbol of `"ard$rcaaaabb"`, then we obtain `"b"` in the second position of *Text* (left matrix below). On the other hand, if we guess that this `"a"` is the ninth symbol of `"ard$rcaaaabb"`, then we obtain `"c"` in the second position of *Text* (right matrix below). Finally, if we guess that this `"a"` is the tenth symbol of `"ard$rcaaaabb"`, then we obtain `"d"` in the second position of *Text* (bottom matrix below).







STOP and Think: Take another look at the three alternatives below. How would you choose among `"b"`, `"c"`, and `"d"` for the second symbol of *Text*?



The First-Last Property

To determine the remaining symbols of *Text*, we need to use a subtle property of *M*(*Text*) that may seem completely unrelated to inverting the Burrows-Wheeler transform. Below, we have indexed the occurrences of each symbol in *FirstColumn* with subscripts according to their order of appearance. When *Text* = "panamabananas$", six instances of "a" appear in *FirstColumn*, as shown below.

```
$  p a n a m a b a n a n a s
a1 b a n a n a s $ p a n a m
a2 m a b a n a n a s $ p a n
a3 n a m a b a n a n a s $ p
a4 n a n a s $ p a n a m a b
a5 n a s $ p a n a m a b a n
a6 s $ p a n a m a b a n a n
b  a n a n a s $ p a n a m a
m  a b a n a n a s $ p a n a
n  a m a b a n a n a s $ p a
n  a n a s $ p a n a m a b a
n  a s $ p a n a m a b a n a
p  a n a m a b a n a n a s $
s  $ p a n a m a b a n a n a
```

Consider "a_1" in *FirstColumn*, which occurs at the beginning of the cyclic rotation "$a_1$bananas$panam". If we cyclically rotate this string, then we obtain "panam$a_1$bananas$". Thus, "$a_1$" in *FirstColumn* is actually the third occurrence of "a" in "panam**a**bananas$". We can easily identify the positions of the other five instances of "a" in "panamabananas$":

$$pa_3na_2ma_1ba_4na_5na_6s\$$$

STOP and Think: Where are the three instances of "n" from *FirstColumn* (i.e., "n_1", "n_2", and "n_3") located in "panamabananas$"?

To locate "$\mathbf{a_1}$" in *LastColumn*, we need to cyclically rotate the second row of *M*("$\mathbf{a_1}$bananas$panam"), which results in "bananas$panam$\mathbf{a_1}$". This rotation corresponds to the eighth row of *M*("panamabananas$"), as shown below.

$ p a n a m a b a n a n a s
$\mathbf{a_1}$ b a n a n a s $ p a n a m
$\mathbf{a_2}$ m a b a n a n a s $ p a n
$\mathbf{a_3}$ n a m a b a n a n a s $ p
$\mathbf{a_4}$ n a n a s $ p a n a m a b
$\mathbf{a_5}$ n a s $ p a n a m a b a n
$\mathbf{a_6}$ s $ p a n a m a b a n a n
b a n a n a s $ p a n a m $\mathbf{a_1}$
m a b a n a n a s $ p a n a
n a m a b a n a n a s $ p a
n a n a s $ p a n a m a b a
n a s $ p a n a m a b a n a
p a n a m a b a n a n a s $
s $ p a n a m a b a n a n a

STOP and Think: Where are the other five instances of "a" located in *LastColumn*?

You hopefully saw that *LastColumn* can be recorded as "smnpbnn$a_1a_2a_3a_4a_5$a_6$":

$	p	a	n	a	m	a	b	a	n	a	n	a	s
a_1	b	a	n	a	n	a	s	$	p	a	n	a	m
a_2	m	a	b	a	n	a	n	a	s	$	p	a	n
a_3	n	a	m	a	b	a	n	a	n	a	s	$	p
a_4	n	a	n	a	s	$	p	a	n	a	m	a	b
a_5	n	a	s	$	p	a	n	a	m	a	b	a	n
a_6	s	$	p	a	n	a	m	a	b	a	n	a	n
b	a	n	a	n	a	s	$	p	a	n	a	m	a_1
m	a	b	a	n	a	n	a	s	$	p	a	n	a_2
n	a	m	a	b	a	n	a	n	a	s	$	p	a_3
n	a	n	a	s	$	p	a	n	a	m	a	b	a_4
n	a	s	$	p	a	n	a	m	a	b	a	n	a_5
p	a	n	a	m	a	b	a	n	a	n	a	s	$
s	$	p	a	n	a	m	a	b	a	n	a	n	a_6

Note that the six instances of "a" appear in exactly the same order in *FirstColumn* and *LastColumn*. This observation is not a fluke. On the contrary, it is a principle that holds for any string *Text* and any symbol that we choose.

First-Last Property: The *k*-th occurrence of a symbol in *FirstColumn* and the *k*-th occurrence of this symbol in *LastColumn* correspond to the same position of this symbol in *Text*.

To see why the First-Last Property must be true, consider the rows of *M*("panamabananas$") beginning with "a":

$\mathbf{a_1}$ b a n a n a s $ p a n a m
$\mathbf{a_2}$ m a b a n a n a s $ p a n
$\mathbf{a_3}$ n a m a b a n a n a s $ p
$\mathbf{a_4}$ n a n a s $ p a n a m a b
$\mathbf{a_5}$ n a s $ p a n a m a b a n
$\mathbf{a_6}$ s $ p a n a m a b a n a n

These rows are already ordered lexicographically, so if we chop off the "a" from the beginning of each row, then the remaining strings should still be ordered lexicographically:

b a n a n a s $ p a n a m
m a b a n a n a s $ p a n
n a m a b a n a n a s $ p
n a n a s $ p a n a m a b
n a s $ p a n a m a b a n
s $ p a n a m a b a n a n

Adding "a" back to the end of each row should not change the lexicographic ordering of these rows:

b a n a n a s $ p a n a m $\mathbf{a_1}$
m a b a n a n a s $ p a n $\mathbf{a_2}$
n a m a b a n a n a s $ p $\mathbf{a_3}$
n a n a s $ p a n a m a b $\mathbf{a_4}$
n a s $ p a n a m a b a n $\mathbf{a_5}$
s $ p a n a m a b a n a n $\mathbf{a_6}$

But these are just the rows of *M*("panamabananas$") containing "a" in *LastColumn*! As a result, the *k*-th occurrence of "a" in *FirstColumn* corresponds to the *k*-th occurrence of "a" in *LastColumn*. This argument generalizes for any *symbol* and any string *Text*, which establishes the First-Last property.

Using the First-Last Property for Burrows-Wheeler inversion

The First-Last Property is interesting, but how can we use it to invert *BWT*(*Text*) = "ard$rcaaaabb"? Let's return to where we were in our attempt to reconstruct the first row of *M*(*Text*) and index the occurrences of each symbol in *FirstColumn* and *LastColumn*:

$$
\begin{array}{cccccccccccc}
\$_1 & a & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_1 \\
a_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & r_1 \\
a_2 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & d_1 \\
a_3 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & \$_1 \\
a_4 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & r_2 \\
a_5 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & c_1 \\
b_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_2 \\
b_2 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_3 \\
c_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_4 \\
d_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_5 \\
r_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & b_1 \\
r_2 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & b_2
\end{array}
$$

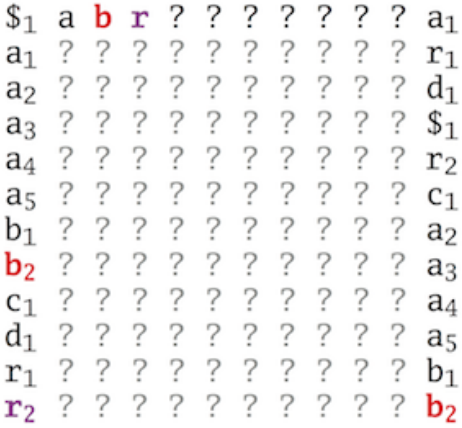
The First-Last Property reveals where "a_3" is hiding in *LastColumn*:

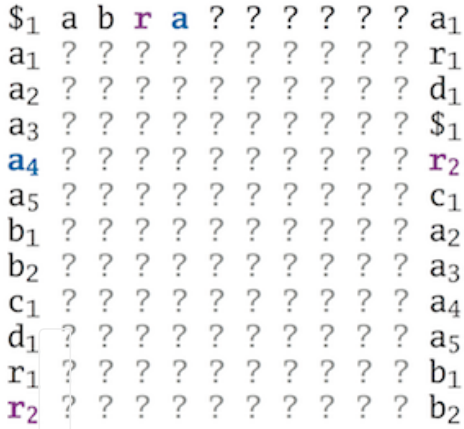
$$
\begin{array}{cccccccccccc}
\$_1 & a & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_1 \\
a_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & r_1 \\
a_2 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & d_1 \\
a_3 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & \$_1 \\
a_4 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & r_2 \\
a_5 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & c_1 \\
b_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_2 \\
b_2 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_3 \\
c_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_4 \\
d_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & a_5 \\
r_1 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & b_1 \\
r_2 & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & ? & b_2
\end{array}
$$

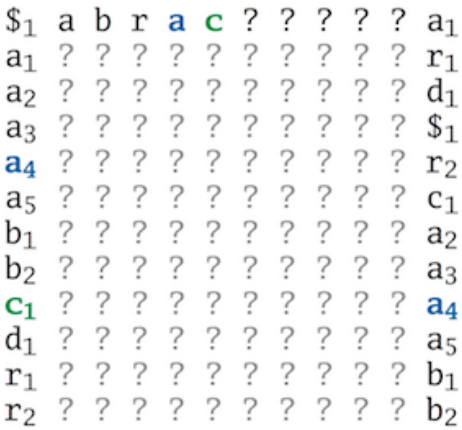
Since we know that "$\mathbf{a_3}$" is located at the end of the eighth row, we can wrap around this row to determine that "$\mathbf{b_2}$" follows "$\mathbf{a_3}$" in *Text*. Thus, the second symbol of *Text* is "**b**", which we can now add to the first row of *M*(*Text*):

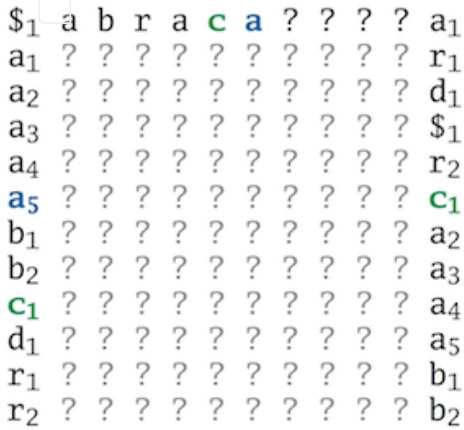
$\$_1$	**a**	**b**	?	?	?	?	?	?	?	?	?	a_1
a_1	?	?	?	?	?	?	?	?	?	?	?	r_1
a_2	?	?	?	?	?	?	?	?	?	?	?	d_1
$\mathbf{a_3}$	?	?	?	?	?	?	?	?	?	?	?	$\$_1$
a_4	?	?	?	?	?	?	?	?	?	?	?	r_2
a_5	?	?	?	?	?	?	?	?	?	?	?	c_1
b_1	?	?	?	?	?	?	?	?	?	?	?	a_2
$\mathbf{b_2}$	?	?	?	?	?	?	?	?	?	?	?	$\mathbf{a_3}$
c_1	?	?	?	?	?	?	?	?	?	?	?	a_4
d_1	?	?	?	?	?	?	?	?	?	?	?	a_5
r_1	?	?	?	?	?	?	?	?	?	?	?	b_1
r_2	?	?	?	?	?	?	?	?	?	?	?	b_2

On this step and the next one, we illustrate repeated applications of the First-Last Property to reconstruct more and more symbols from *Text*.









Presto — the string that we have been trying to reconstruct is `"abracadabra$"`.

$_1$	a	b	r	a	c	a	d	?	?	?	a$_1$
a$_1$	?	?	?	?	?	?	?	?	?	?	r$_1$
a$_2$	?	?	?	?	?	?	?	?	?	?	d$_1$
a$_3$	?	?	?	?	?	?	?	?	?	?	$_1$
a$_4$	?	?	?	?	?	?	?	?	?	?	r$_2$
a$_5$	?	?	?	?	?	?	?	?	?	?	c$_1$
b$_1$	?	?	?	?	?	?	?	?	?	?	a$_2$
b$_2$	?	?	?	?	?	?	?	?	?	?	a$_3$
c$_1$	?	?	?	?	?	?	?	?	?	?	a$_4$
d$_1$	?	?	?	?	?	?	?	?	?	?	a$_5$
r$_1$	?	?	?	?	?	?	?	?	?	?	b$_1$
r$_2$	?	?	?	?	?	?	?	?	?	?	b$_2$

$_1$	a	b	r	a	c	a	d	a	?	?	a$_1$
a$_1$	?	?	?	?	?	?	?	?	?	?	r$_1$
a$_2$	?	?	?	?	?	?	?	?	?	?	d$_1$
a$_3$	?	?	?	?	?	?	?	?	?	?	$_1$
a$_4$	?	?	?	?	?	?	?	?	?	?	r$_2$
a$_5$	?	?	?	?	?	?	?	?	?	?	c$_1$
b$_1$	?	?	?	?	?	?	?	?	?	?	a$_2$
b$_2$	?	?	?	?	?	?	?	?	?	?	a$_3$
c$_1$	?	?	?	?	?	?	?	?	?	?	a$_4$
d$_1$	?	?	?	?	?	?	?	?	?	?	a$_5$
r$_1$	?	?	?	?	?	?	?	?	?	?	b$_1$
r$_2$	?	?	?	?	?	?	?	?	?	?	b$_2$

$_1$	a	b	r	a	c	a	d	a	b	?	a$_1$
a$_1$	?	?	?	?	?	?	?	?	?	?	r$_1$
a$_2$	?	?	?	?	?	?	?	?	?	?	d$_1$
a$_3$	?	?	?	?	?	?	?	?	?	?	$_1$
a$_4$	?	?	?	?	?	?	?	?	?	?	r$_2$
a$_5$	?	?	?	?	?	?	?	?	?	?	c$_1$
b$_1$	?	?	?	?	?	?	?	?	?	?	a$_2$
b$_2$	?	?	?	?	?	?	?	?	?	?	a$_3$
c$_1$	?	?	?	?	?	?	?	?	?	?	a$_4$
d$_1$	?	?	?	?	?	?	?	?	?	?	a$_5$
r$_1$	?	?	?	?	?	?	?	?	?	?	b$_1$
r$_2$	?	?	?	?	?	?	?	?	?	?	b$_2$

$_1$	a	b	r	a	c	a	d	a	b	r	a$_1$
a$_1$	?	?	?	?	?	?	?	?	?	?	r$_1$
a$_2$	?	?	?	?	?	?	?	?	?	?	d$_1$
a$_3$	?	?	?	?	?	?	?	?	?	?	$_1$
a$_4$	?	?	?	?	?	?	?	?	?	?	r$_2$
a$_5$	?	?	?	?	?	?	?	?	?	?	c$_1$
b$_1$	?	?	?	?	?	?	?	?	?	?	a$_2$
b$_2$	?	?	?	?	?	?	?	?	?	?	a$_3$
c$_1$	?	?	?	?	?	?	?	?	?	?	a$_4$
d$_1$	?	?	?	?	?	?	?	?	?	?	a$_5$
r$_1$	?	?	?	?	?	?	?	?	?	?	b$_1$
r$_2$	?	?	?	?	?	?	?	?	?	?	b$_2$

EXERCISE BREAK: Reconstruct the string whose Burrows-Wheeler transform is `"enwvpeoseu$llt"`. (Don't forget the `$` at the end!)

To solve this problem please visit stepic.org/lesson/-299/step/9

You are now ready to implement the inverse of the Burrows-Wheeler transform.

Inverse Burrows-Wheeler Transform Problem: *Reconstruct a string from its Burrows-Wheeler transform.*
Input: A string *Transform* (with a single `"$"` symbol).
Output: The string *Text* such that *BWT*(*Text*) = *Transform*.

CODE CHALLENGE: Solve the Inverse Burrows-Wheeler Transform Problem.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/InverseBWT.txt)

Sample Input:

```
TTCCTAACG$A
```

Sample Output:

```
TACATCACGT$
```

Time Limit: `5 mins`

To solve this problem please visit stepic.org/lesson/-299/step/10

STOP and Think: Can any string (having a single "$" symbol) be inverted using the inverse Burrows-Wheeler transform?

A first attempt at Burrows-Wheeler pattern matching

The Burrows-Wheeler transform may be fascinating, but how could it possibly help us decrease the memory required for pattern matching?

The idea motivating a Burrows-Wheeler-based approach to pattern matching relies on the observation that each row of *M*(*Text*) begins with a different suffix of *Text*. Since these suffixes are already ordered lexicographically, any matches of *Pattern* in *Text* will appear at the beginning of consecutive rows of *M*(*Text*), as shown in the figure below.

Figure: Because the rows of *M*(*Text*) are ordered lexicographically, suffixes beginning with the same string (`"ana"`) clump together in the matrix.

We now have the outline of a method to match *Pattern* to *Text*. Construct *M*(*Text*), and identify rows beginning with the first symbol of *Pattern*. Among these rows, determine which ones have a second element matching the second symbol of *Pattern*. Continue this process until we find which rows of *M*(*Text*) begin with *Pattern*.

STOP and Think: What is wrong with this approach?

Moving backward through a pattern

The issue with this proposed method for pattern matching is that we cannot afford storing the entire matrix M(Text), which has $|Text|^2$ entries. In an effort to reduce memory requirements, let's forbid ourselves from accessing any information in *M*(*Text*) other than *FirstColumn* and *LastColumn*. Using these two columns, we will try to match *Pattern* to *Text* by moving backward through *Pattern*. For example, if we want to match *Pattern* = `"ana"` to *Text* = `"panamabananas$"`, then we will first identify rows of *M*(*Text*) beginning with `"a"`, the last letter of `"ana"`:

$_1$ p a n a m a b a n a n a s$_1$
a$_1$ b a n a n a s $ p a n a m$_1$
a$_2$ m a b a n a n a s $ p a n$_1$
a$_3$ n a m a b a n a n a s $ p$_1$
a$_4$ n a n a s $ p a n a m a b$_1$
a$_5$ n a s $ p a n a m a b a n$_2$
a$_6$ s $ p a n a m a b a n a n$_3$
b$_1$ a n a n a s $ p a n a m a$_1$
m$_1$ a b a n a n a s $ p a n a$_2$
n$_1$ a m a b a n a n a s $ p a$_3$
n$_2$ a n a s $ p a n a m a b a$_4$
n$_3$ a s $ p a n a m a b a n a$_5$
p$_1$ a n a m a b a n a n a s $$_1$
s$_1$ $ p a n a m a b a n a n a$_6$

As we are moving backward through "ana", we will next look for rows of $M(Text)$ beginning with "na". To do this without knowing the entire matrix $M(Text)$, we again use the fact that a symbol in *LastColumn* must precede the symbol of *Text* found in the same row in *FirstColumn*. Thus, we only need to identify those rows of $M(Text)$ beginning with "a" and ending with "n":

$$
\begin{array}{l}
\$_1\ p\ a\ n\ a\ m\ a\ b\ a\ n\ a\ n\ a\ s_1 \\
a_1\ b\ a\ n\ a\ n\ a\ s\ \$\ p\ a\ n\ a\ m_1 \\
\mathbf{a_2}\ m\ a\ b\ a\ n\ a\ n\ a\ s\ \$\ p\ a\ \mathbf{n_1} \\
a_3\ n\ a\ m\ a\ b\ a\ n\ a\ n\ a\ s\ \$\ p_1 \\
a_4\ n\ a\ n\ a\ s\ \$\ p\ a\ n\ a\ m\ a\ b_1 \\
\mathbf{a_5}\ n\ a\ s\ \$\ p\ a\ n\ a\ m\ a\ b\ a\ \mathbf{n_2} \\
\mathbf{a_6}\ s\ \$\ p\ a\ n\ a\ m\ a\ b\ a\ n\ a\ \mathbf{n_3} \\
b_1\ a\ n\ a\ n\ a\ s\ \$\ p\ a\ n\ a\ m\ a_1 \\
m_1\ a\ b\ a\ n\ a\ n\ a\ s\ \$\ p\ a\ n\ a_2 \\
n_1\ a\ m\ a\ b\ a\ n\ a\ n\ a\ s\ \$\ p\ a_3 \\
n_2\ a\ n\ a\ s\ \$\ p\ a\ n\ a\ m\ a\ b\ a_4 \\
n_3\ a\ s\ \$\ p\ a\ n\ a\ m\ a\ b\ a\ n\ a_5 \\
p_1\ a\ n\ a\ m\ a\ b\ a\ n\ a\ n\ a\ s\ \$_1 \\
s_1\ \$\ p\ a\ n\ a\ m\ a\ b\ a\ n\ a\ n\ a_6
\end{array}
$$

The First-Last Property tells us where to find the three highlighted "n" in *FirstColumn*, as shown below. All three rows end with "a", yielding three total occurrences of "ana" in *Text*.

$$
\begin{array}{l}
\$_1\ p\ a\ n\ a\ m\ a\ b\ a\ n\ a\ n\ a\ s_1 \\
a_1\ b\ a\ n\ a\ n\ a\ s\ \$\ p\ a\ n\ a\ m_1 \\
a_2\ m\ a\ b\ a\ n\ a\ n\ a\ s\ \$\ p\ a\ n_1 \\
a_3\ n\ a\ m\ a\ b\ a\ n\ a\ n\ a\ s\ \$\ p_1 \\
a_4\ n\ a\ n\ a\ s\ \$\ p\ a\ n\ a\ m\ a\ b_1 \\
a_5\ n\ a\ s\ \$\ p\ a\ n\ a\ m\ a\ b\ a\ n_2 \\
a_6\ s\ \$\ p\ a\ n\ a\ m\ a\ b\ a\ n\ a\ n_3 \\
b_1\ a\ n\ a\ n\ a\ s\ \$\ p\ a\ n\ a\ m\ a_1 \\
m_1\ a\ b\ a\ n\ a\ n\ a\ s\ \$\ p\ a\ n\ a_2 \\
\mathbf{n_1}\ \mathbf{a}\ m\ a\ b\ a\ n\ a\ n\ a\ s\ \$\ p\ \mathbf{a_3} \\
\mathbf{n_2}\ \mathbf{a}\ n\ a\ s\ \$\ p\ a\ n\ a\ m\ a\ b\ \mathbf{a_4} \\
\mathbf{n_3}\ \mathbf{a}\ s\ \$\ p\ a\ n\ a\ m\ a\ b\ a\ n\ \mathbf{a_5} \\
p_1\ a\ n\ a\ m\ a\ b\ a\ n\ a\ n\ a\ s\ \$_1 \\
s_1\ \$\ p\ a\ n\ a\ m\ a\ b\ a\ n\ a\ n\ a_6
\end{array}
$$

The highlighted occurrences of "a" in *LastColumn* correspond to the third, fourth, and fifth occurrences of `"a"` in this column, and the First-Last Property tells us that they should correspond to the third, fourth, and fifth occurrences of `"a"` in *FirstColumn* as well, which identifies the three matches of `"ana"`:

$_1$ p a n a m a b a n a n a s$_1$
a$_1$ b a n a n a s $ p a n a m$_1$
a$_2$ m a b a n a n a s $ p a n$_1$
a$_3$ n a m a b a n a n a s $ p$_1$
a$_4$ n a n a s $ p a n a m a b$_1$
a$_5$ n a s $ p a n a m a b a n$_2$
a$_6$ s $ p a n a m a b a n a n$_3$
b$_1$ a n a n a s $ p a n a m a$_1$
m$_1$ a b a n a n a s $ p a n a$_2$
n$_1$ a m a b a n a n a s $ p a$_3$
n$_2$ a n a s $ p a n a m a b a$_4$
n$_3$ a s $ p a n a m a b a n a$_5$
p$_1$ a n a m a b a n a n a s $$_1$
s$_1$ $ p a n a m a b a n a n a$_6$

EXERCISE BREAK: Match *Pattern* = `"banana"` to *Text* = `"panamabananas$"` by walking backward through *Pattern* using the Burrows-Wheeler transform of *Text*.

Burrows-Wheeler pattern matching with pointers

We now know how to use *BWT*(*Text*) to find all matches of *Pattern* in *Text* by walking backward through *Pattern*. However, every time we walk backward, we need to keep track of where the matches of a suffix of *Pattern* are hiding at the beginning of *M*(*Text*). Fortunately, we know that at each step, the rows of *M*(*Text*) that match a suffix of Pattern clump together in consecutive rows of *M*(*Text*). This means that the collection of all matching rows is revealed by only two pointers, *top* and *bottom*: *top* holds the index of the first row of *M*(*Text*) that matches the current suffix of *Pattern*, and *bottom* holds the index of the last row of *M*(*Text*) that matches this suffix. The figure below shows the process of updating pointers; after walking backward through *Pattern* = `"ana"`, we have that *top* = 3 and *bottom* = 5. After traversing *Pattern*, we can compute the total number of matches of *Pattern* in *Text* by calculating *bottom* − *top* + 1 (e.g., there are 5 − 3 + 1 = 3 matches of `"ana"` in `"panamabananas$"`).

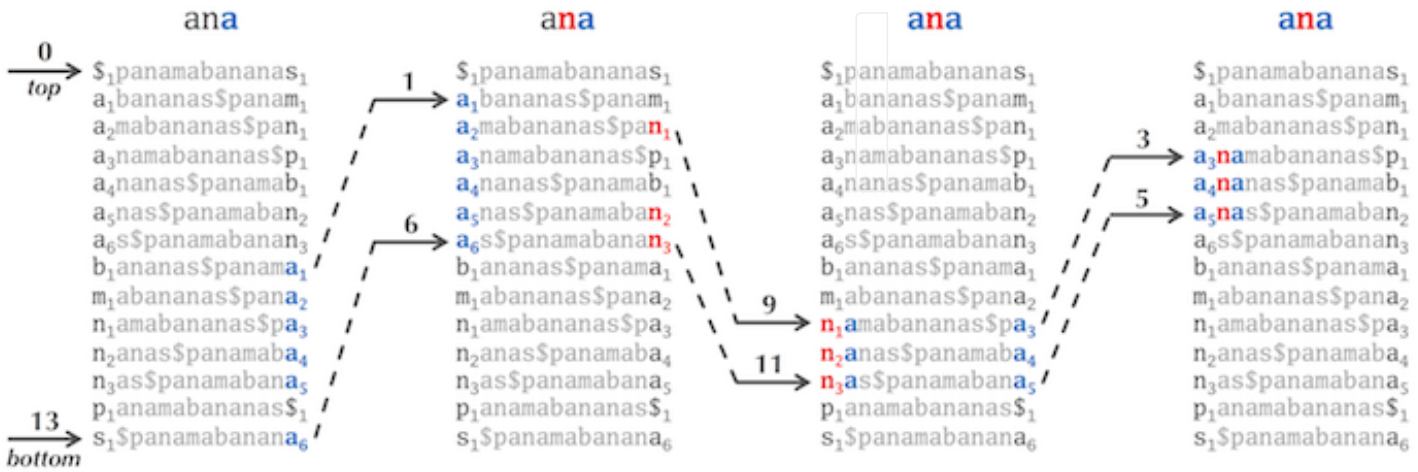


Figure: The pointers *top* and *bottom* hold the indices of the first and last rows of *M*(*Text*) matching the current suffix of *Pattern*. The above diagram shows how these pointers are updated when walking backwards through `"ana"` and looking for substring matches in `"panamabananas$"`.

Let's concentrate on how pointers are updated from one stage to the next. Consider the transition from the second to the third panel in the figure above; how did we know to update the pointers (*top* = 1, *bottom* = 6) into (*top* = 9, *bottom* = 11)? We are looking for the first and last occurrence of `"n"` in the range of positions from *top* = 1 to *bottom* = 6 in *LastColumn*. The first occurrence of `"n"` in this range is "$\mathbf{n_1}$" (in position 2) and the last is "$\mathbf{n_3}$" (position 6).

In order to update the *top* and *bottom* pointers, we need to determine where "n_1" and "n_3" occur in *FirstColumn*. The **Last-to-First array**, denoted *LastToFirst*(*i*), answers the following question: given a symbol at position *i* in *LastColumn*, what is its position in *FirstColumn*? For our ongoing example, *LastToFirst*(2) = 9, since the symbol at position 2 of *LastColumn* ("n_1") corresponds to position 9 in *FirstColumn*, as shown in the table below. Similarly, *LastToFirst*(6) = 11, since the symbol at position 6 of *LastColumn* ("n_3") occurs at position 11 in *FirstColumn*. Therefore, with the help of the Last-to-First mapping, we can quickly update the pointers (*top* = 1, *bottom* = 6) into (*top* = 9, *bottom* = 11).

i	*FirstColumn*	*LastColumn*	LastToFirst(*i*)
0	$\$_1$	s_1	13
1	a_1	m_1	8
2	a_2	n_1	9
3	a_3	p_1	12
4	a_4	b_1	7
5	a_5	n_2	10
6	a_6	n_3	11
7	b_1	a_1	1
8	m_1	a_2	2
9	n_1	a_3	3
10	n_2	a_4	4
11	n_3	a_5	5
12	p_1	$\$_1$	0
13	s_1	a_6	6

Figure: Given a symbol at position *i* of *LastColumn*, the Last-to-First mapping identifies this symbol's position in *FirstColumn*.

We are now ready to describe **BWMATCHING**, an algorithm that counts the total number of matches of *Pattern* in *Text*, where the only information that we are given is *FirstColumn* and *LastColumn* in addition to the Last-to-First mapping. The pointers *top* and *bottom* are updated by the green lines in the following pseudocode.

```
BWMATCHING(FirstColumn, LastColumn, Pattern, LastToFirst)
  top ← 0
  bottom ← |LastColumn| − 1
  while top ≤ bottom
    if Pattern is nonempty
      symbol ← last letter in Pattern
      remove last letter from Pattern
      if positions from top to bottom in LastColumn contain an occurrence of symbol
        topIndex ← first position of symbol among positions from top to bottom in LastColumn
        bottomIndex ← last position of symbol among positions from top to bottom in LastColumn
        top ← LastToFirst(topIndex)
        bottom ← LastToFirst(bottomIndex)
      else
        return 0
    else
      return bottom − top + 1
```

CODE CHALLENGE: Implement **BWMATCHING**.

Input: A string *BWT*(*Text*), followed by a collection of *Patterns*.

Output: A list of integers, where the *i*-th integer corresponds to the number of substring matches of the *i*-th member of *Patterns* in *Text*.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/BWMatching.txt)

Sample Input:

```
TCCTCTATGAGATCCTATTCTATGAAACCTTCA$GACCAAAATTCTCCGGC
CCT CAC GAG CAG ATC
```

Sample Output:

```
2 1 1 0 1
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-300/step/8

We can obtain *SuffixArray*(*Text*) from *SuffixTree*(*Text*) by applying a **preorder traversal** of *SuffixTree*(*Text*) as long as the outgoing edges from every node of the suffix tree are arranged lexicographically.

Given a rooted tree, a node *w* is called a **child** of a node *v* if there is an edge connecting *v* to *w* in the tree. The preorder traversal of a tree involves visiting a node of the tree, starting at the root, and then recursively preorder traversing the subtrees rooted at each of its children from left to right (see figure below). By taking the order of leaves visited in a preorder traversal of the suffix tree, we obtain the suffix array.

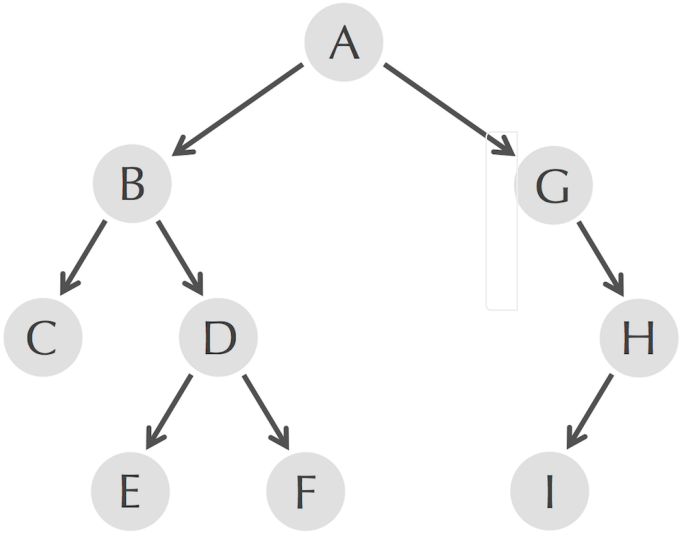


Figure: The preorder traversal of the above tree visits its nodes in increasing order of their labels.

The preorder traversal is accomplished by the following pseudocode.

```
PREORDER(Tree, Node)
    visit Node
    for each child Node' of Node from left to right
        PREORDER(Tree, Node')
```

Conversely, *SuffixTree*(*Text*) can be constructed in linear time from *SuffixArray*(*Text*) by using the **longest common prefix (LCP) array** of Text, *LCP*(*Text*), which stores the length of the longest common prefix shared by consecutive lexicographically ordered suffixes of *Text*. For example, LCP("panamabananas$") is (0, 0, 1, 1, 3, 3, 1, 0, 0, 0, 2, 2, 0, 0), as shown below.

LCP Array	Sorted Suffixes
0	$
0	abananas$
1	| amabananas$
1	| anamabananas$
3	||| ananas$
3	||| anas$
1	| as$
0	bananas$
0	mabananas$
0	namabananas$
2	|| nanas$
2	|| nas$
0	panamabananas$
0	s$

Suffix Tree Construction from Suffix Array Problem: *Construct a suffix tree from the suffix array and LCP array of a string.*

Input: A string *Text*, its suffix array, and its LCP array.

Output: The suffix tree of *Text*.

Given the suffix array *SuffixArray* and LCP array *LCP* of a string *Text*, the suffix tree *SuffixTree*(*Text*) can be constructed in linear time using the algorithm illustrated in the figure on the next step. After constructing a **partial suffix tree** for the *i* lexicographically smallest suffixes (denoted $SuffixTree_i(Text)$), this algorithm iteratively inserts the (*i* + 1)-th suffix into this tree to form (denoted $SuffixTree_{i+1}(Text)$).

We define the **descent** of a leaf *v* in a suffix tree, denoted *Descent*(*v*), as the length of the concatenation of all path labels from the root to this leaf. We assume that the descents of all nodes in the growing partial suffix tree have been precomputed.

We start with $SuffixTree_0(Text)$, which we define as the tree consisting only of the root. To insert the (*i* + 1)-th suffix (corresponding to the element *SuffixArray*(*i*+1) in the suffix array of *Text*) into $SuffixTree_i(Text)$, we need to know where the path representing this suffix "splits" from the already constructed partial suffix tree $SuffixTree_i(Text)$. As shown in the figure on the next step, this split happens at the purple node while inserting `"anamabananas$"` into $SuffixTree_3(Text)$ and at the purple edge labeled `"namabananas$"` while inserting `"ananas$"` into $SuffixTree_4(Text)$, thus breaking this edge into edges labeled `"na"` and `"mabananas$"` in $SuffixTree_5(Text)$.

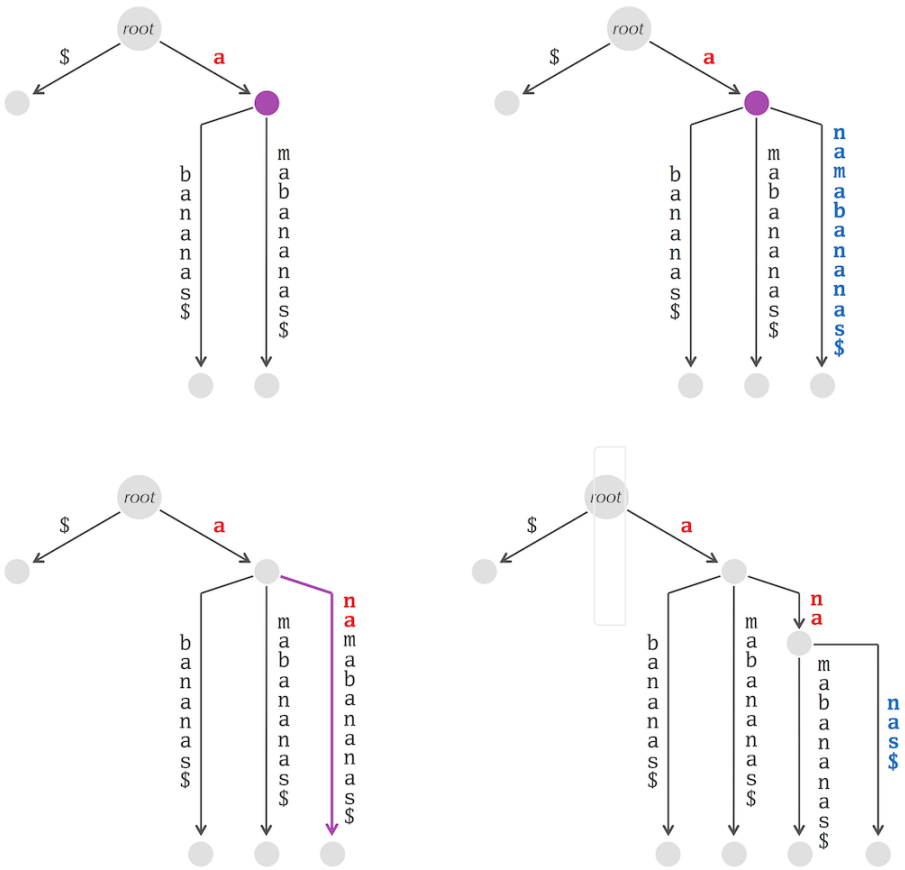


Figure: (Top left) *SuffixTree*$_3$(*Text*) for *Text* = "panamabananas$". The fourth lexicographically ordered suffix of *Text* is "**anamabananas$**", and we can thread only the first letter into this suffix tree. Our stopping point is at the purple node, so we create a new node branching off this node with edge labeled "namabananas$" to obtain *SuffixTree*$_4$(*Text*) (top right). (Bottom left) The fifth lexicographically ordered suffix of *Text* is "**ananas$**", the first three symbols of which we can thread into *SuffixTree*$_4$(*Text*) until we reach a stopping point in the middle of the edge "**na**mabananas$". (Bottom right) To form *SuffixTree*$_5$(*Text*) , we branch at the node following the purple edge into two edges: one labeled "mabananas$" in order to retain the suffix "anamabananas$" and another labeled "**nas$**" to spell the new suffix "**ananas$**". In general, the number of red symbols that we spell down in *SuffixTree*$_i$(*Text*) to form *SuffixTree*$_{i+1}$(*Text*) is given by the (*i* + 1)-th entry in the LCP array of *Text*.

To find the node/edge where the partial suffix tree splits, we will walk up the rightmost path (i.e., the last added path in the partial suffix tree) in the partial suffix tree beginning at the previously inserted leaf, labeled *SuffixArray*(*i* + 1), to the root. We stop when we encounter the first node *v* such that *Descent*(*v*) ≤ *LCP*(*i* + 1). Afterwards, we have to consider two cases depending on whether *Descent*(*v*) = *LCP*(*i* + 1) (the split occurs at the node *v*) or *Descent*(*v*) < *LCP*(*i* + 1) (the split occurs on the edge leading from *v*):

If *Descent*(*v*) = *LCP*(*i* + 1), then the concatenation of the labels on the path from the root to v equals the longest common prefix of suffixes corresponding to *SuffixArray*(*i*) and *SuffixArray*(*i* + 1). In this case, we insert *SuffixArray*(*i* + 1) as a new leaf *x* connected to *v*, and we label the edge (*v*, *x*) with the suffix of *Text* starting at position *SuffixArray*(*i* + 1) + *LCP*(*i* + 1). Thus, the edge label consists of the remaining symbols of the suffix corresponding to *SuffixArray*(*i* + 1) that are not already represented by the concatenation of the labels of the path connecting the root to *v*. This completes the construction of the partial suffix tree $SuffixTree_{i+1}(Text)$.

If *Descent*(*v*) = *LCP*(*i* + 1), then the concatenation of the labels on the path from the root to *v* has fewer symbols than the longest common prefix of the suffixes corresponding to *SuffixArray*(*i*) and *SuffixArray*(*i* + 1). The question therefore arises of how to recover these missing symbols. We denote the edge on the rightmost edge leading from *v* in $SuffixTree_i(Text)$ as (*v*, *w*) and argue that the missing symbols are a prefix of this edge's label. In this case, we will split this edge and construct $SuffixTree_{i+1}(Text)$ as follows:

1. Delete (*v*, *w*) from $SuffixTree_i(Text)$.
2. Add a new internal node *y* and a new edge (*v*, *y*) labeled by a substring of *Text* starting at position *SuffixArray*(*i*+1) + *Descent*(*v*) and ending at position *SuffixArray*(*i*) + *LCP*(*i* + 1) - 1. The new label consists of the missing symbols of the longest common prefix of *SuffixArray*(*i*) and *SuffixArray*(*i* + 1). Thus, the concatenation of the labels on the path from the root to *y* is now the longest common prefix of *SuffixArray*(*i*) and *SuffixArray*(*i* + 1).
3. Define *Descent*(*y*) as *SuffixArray*(*i* + 1).
4. Connect *w* to the newly created internal node *y* by an edge (*y*, *w*) that is labeled by a substring of *Text* starting at position *SuffixArray*(*i*) + *LCP*(*i* + 1) and ending at position *SuffixArray*(*i*) + *Descent*(*w*) - 1. The new label consists of the remaining symbols of the deleted edge (*v*, *w*) that were not used as the label of edge (*v*, *y*).
5. Add *SuffixArray*(*i* + 1) as a new leaf *x* as well as an edge (*y*, *x*) that is labeled by a suffix of *Text* beginning at position *SuffixArray*(*i* + 1) + *LCP*(*i* + 1). Thus, the edge label consists of the remaining symbols of the suffix corresponding to *SuffixArray*(*i* + 1) that are not already represented by the concatenation of the labels on the path from the root to *v*.

STOP and Think: Prove that the running time of this algorithm is O(|*Text*|).

CODE CHALLENGE: Solve the Constructing Suffix Tree from Suffix Array Problem (restated below). As in the Suffix Tree Construction Problem, you need only return the list of all strings forming edge labels of the suffix tree.

Constructing Suffix Tree from Suffix Array Problem: *Construct a suffix tree from the suffix array and LCP array of a string.*
Input: A string *Text*, *SuffixArray*(*Text*), and *LCP*(*Text*).
Output: *SuffixTree*(*Text*).

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/bowtie/SuffixTreeFromSuffixArray.txt)

Return to Main Text (https://stepic.org/lesson/Suffix-Arrays-310/step/4)

Sample Input:

```
GTAGT$
5, 2, 3, 0, 4, 1
0, 0, 0, 2, 0, 1
```

Sample Output:

```
$
T
AGT$
$
AGT$
GT
$
AGT$
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-308/step/8